

S3M Security Staj Ödev Raporu

Ömer Faruk BAYSAL

28 Mart 2026

1. Proje Amacı ve Kapsamı

Bu rapor, S3M Security staj değerlendirme ödevi kapsamında geliştirilen Network Access Control (NAC) sisteminin tüm teknik detaylarını, geliştirme sürecini ve karşılaşılan sorunları kapsamaktadır.

Amaç

Bu projenin temel amacı, ağ güvenliğini sağlamak ve cihaz erişimlerini merkezi olarak yönetmek için RADIUS protokolü tabanlı bir Network Access Control (NAC) sistemi geliştirmektir. Bu sistem sadece şifre sormaz, kimlik doğrulama, yetkilendirme (VLAN ataması) ve hesap yönetimi (accounting) ile birlikte üç temel AAA bileşenini sağlar. Bu projede yapılan çalışmayla birlikte sadece çalışan bir araç değil, kimlik doğrulama politikalarının dışarıdan bir API (FastAPI) ile dinamik olarak yönetilebildiği modern bir mimari de tasarlanmıştır.

Kapsam

Tüm sistem Docker Compose ile orkestre edilen dört servisten oluşturulmuştur.

FreeRADIUS (RADIUS protokolü), FastAPI (policy engine), PostgreSQL (veritabanı) ve Redis (cache ve rate-limiting).

Kullanıcı kimlik doğrulama bcrypt algoritması ile FastAPI üzerinden yapılmaktadır.

Yetkilendirme VLAN attribute'leri dönülerek gerçekleştirilmektedir. Oturum verileri ise hem PostgreSQL'e yazılmakta hem de Redis'te önbelleklenmektedir. Ayrıca Redis'te rate-limit kontrolü de yapılmaktadır.

Proje kapsamında güvenlik standartlarını yükseltmek adına, şifrelerin bcrypt ile hashlendiği ve doğrulamanın API katmanına çekildiği bir yapı kurgulanmıştır.

2. Sistem Mimarisi

Servis	Image	Port	Rol
postgres	postgres:18-alpine	5432	Kullanıcı, grup ve oturum veritabanı
redis	redis:8-alpine	6379	Rate-limit sayacı, aktif oturum cache
api	python:3.13-slim	8000	FastAPI policy engine, bcrypt auth
freeradius	freeradius/freeradius-server:latest-3.2	1812/1813	RADIUS auth/authz/accounting sunucusu

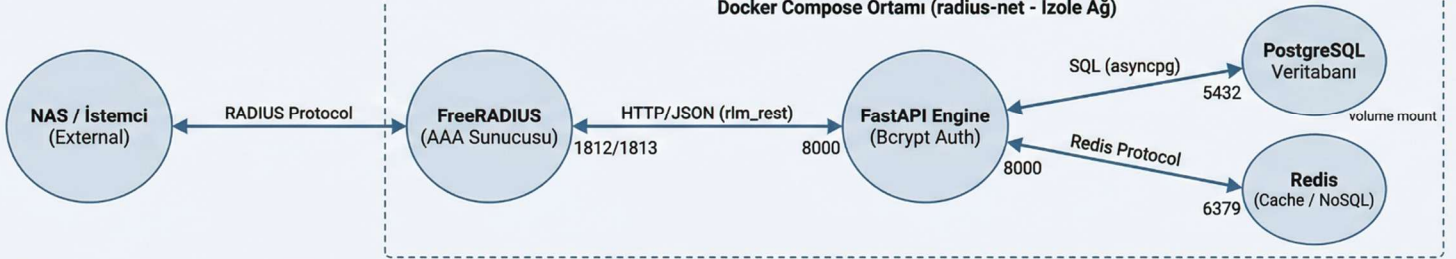
Tüm servisler tek bir docker-compose.yml dosyasında tanımlanmıştır. Her servis için healthcheck, volume mount ve dedicated network konfigürasyonu yapılmıştır.

Tüm sistemin merkezinde FreeRADIUS yer almaktadır. Karar mekanizması olarak FastAPI, verilerin tutulduğu yer ise PostgreSQL ve Redis servisleridir. Tüm bu servisler Docker üzerinde radius-net adlı izole bir ağ üzerinden haberleşmektedir.

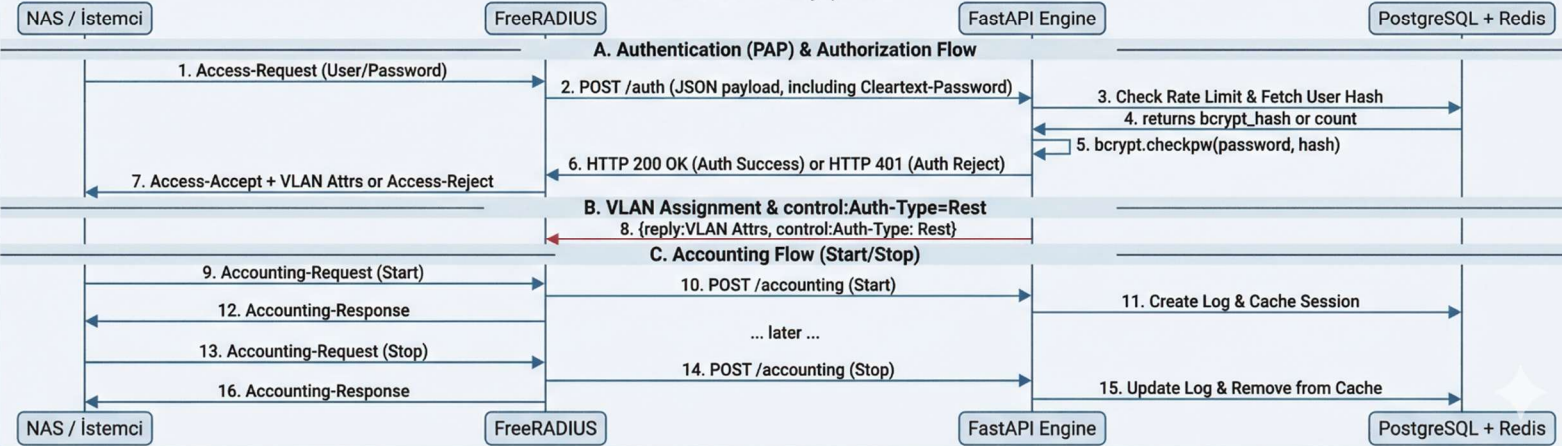
NAC Sistem Mimarisi ve Veri Akışı

Mimari Tasarım Şeması

Docker Compose Ortamı (radius-net - İzole Ağ)



AAA Veri Akış Şeması



Görsel üretiminde yapay zekadan yararlanılmıştır.

AAA Akışı

Authentication aşamasında NAS cihazı FreeRADIUS'a Access-Request gönderir. FreeRADIUS rlm_rest modulu aracılığıyla FastAPI /auth endpoint'ine POST atar. FastAPI kullanıcısının şifresini bcrypt ile doğrular ve Accept/Reject kararını döndürür.

Authorization aşamasında Auth başarılıysa FreeRADIUS /authorize endpoint'ine POST atar. FastAPI kullanıcının grubuna göre Tunnel-Type, Tunnel-Medium-Type ve Tunnel-Private-Group-Id attribute'lerini döndürür. Bu değerler NAS cihazına iletilir ve kullanıcı ilgili VLAN'a atanır.

Son olarak Accounting aşamasındaysa NAS cihazı oturum başlangıcında, arasında ve sonunda Accounting-Request paketleri gönderir. FreeRADIUS bunları /accounting endpoint'ine iletir. FastAPI Start paketinde PostgreSQL'e yeni kayıt açar ve Redis'e aktif oturum olarak ekler. Stop paketinde PostgreSQL kaydını kapatır ve Redis'ten siler.

VLAN Atamaları

Grup	VLAN ID	Tunnel-Private-Group-Id
admin	10	10
employee	20	20
guest	30	30

Veritabanı Şeması

Veritabanı seçiminde asyncpg ile SQL'i kendim oluşturdum. Bu daha kontrollü ve hızlı kod yazımını sağladı. Ayrıca parametrelili yazım imkanı da güvenlik açısından önemli.

PostgreSQL veritabanında yedi adet tablo yer almaktadır. users tablosu bcrypt hash'li şifreleri ve grup bilgisini saklar. radcheck tablosu FreeRADIUS rlm_sql şema uyumluluğu için var olmakla birlikte şifre saklamak için kullanılmamaktadır.

radusergroup kullanıcı-grup ilişkilerini, radgroupreply grup bazlı VLAN attribute'lerini, radacct oturum kayıtlarını, radreply kullanıcıya dönülecek attribute'leri, nas ise NAS cihaz bilgilerini saklar.

Şifreleme

Şifre yönetiminde bcrypt algoritması tercih edilmiştir. FreeRADIUS'un native desteklediği MD5 yerine bcrypt kullanılması, kimlik doğrulamasının tamamen FastAPI üzerinden yapılmasını zorunlu kılmıştır.

Bu yaklaşım güvenlik açısından daha sağlamdır. Çünkü cost parametresi sayesinde hesaplama maliyeti ayarlanabilmekte, her kullanıcı için benzersiz salt üretilabilmekte ve rainbow table saldırıları etkisiz bırakılabilmektedir.

Özetle şifre doğrulama FastAPI üzerinden yapılmaktadır. FastAPI ise kullanıcı kendi tablosundan bcrypt ile doğrular. Accept-Reject kararı da FreeRADIUS üzerinden döndürülür.

3.Uygulama Detayları

Proje Yapısı

- db.py: asyncpg veritabanı bağlantı havuzunu (connection pool) ve Redis bağlantısını yönetir. Bağlantıların yaşam döngüsü, kaynakların verimli kullanılması adına FastAPI'nin lifespan context manager yapısıyla kontrol edilmektedir.
- models.py: FreeRADIUS'un karmaşık liste ve obje formatında gönderdiği özniteliklerin (attribute) düz metne indirgenmesi için FreeRadiusBaseModel adında bir temel sınıf (base class) kurgulanmıştır.
- routes/auth.py: Bcrypt ile şifre doğrulama ve Redis üzerinden hız sınırlandırması (rate-limiting) işlemlerini üstlenir. Art arda 5 başarısız giriş denemesi yapıldığında, ilgili kullanıcı güvenlik amacıyla 300 saniye boyunca engellenir.
- routes/authorize.py: Doğrulanmış kullanıcının dâhil olduğu gruba bakarak uygun VLAN özniteliklerini belirler ve FreeRADIUS'a döndürür.
- routes/accounting.py: Oturum döngüsüne ait Start, Interim-Update ve Stop paketlerini işleyerek PostgreSQL ve Redis üzerindeki oturum (session) verilerini eşzamanlı olarak günceller.

FreeRADIUS Entegrasyonu

FreeRADIUS'un rlm_rest modülü, verileri JSON olarak gönderirken standart bir anahtar-değer yapısı yerine {"type": "string", "value": ["admin1"]} şeklinde liste içeren özel bir format kullanır. Pydantic modellerinde tanımlanan model_validator yapısı ile bu karmaşık yapı yakalanarak doğrudan düz metne (string) dönüştürülmüştür.

Bunun yanı sıra, API uç noktalarından dönen JSON yanıtlarında control:Auth-Type ve reply:Tunnel-* gibi özel ön ekler kullanılmıştır. Bu ön ekler, FreeRADIUS'a ilgili özniteliklerin hangi işlem katmanına (control veya reply) yazılması gerektiğini açıkça bildirerek sistemin yönlendirilmesini sağlar.

FreeRADIUS Konfigürasyonu

- mods-enabled/rest: FastAPI sunucusunun temel adresi ve her AAA aşaması için gerekli olan uç noktalar (endpoint) burada tanımlanmıştır. Bağlantı adresi (connect_uri) olarak Docker ağı içindeki api konteynerinin adresi gösterilmiştir.
- mods-enabled/sql: PostgreSQL veritabanı bağlantı ayarlarını içerir. Hesap yönetimi (accounting) kayıtları için FreeRADIUS'un varsayılan SQL sorgu şablonlarını kullanmak yerine doğrudan FastAPI'nin /accounting uç noktası tercih edilmiştir. Bu mimari karar sayesinde, radacct tablosunda

FreeRADIUS'un zorunlu kıldığı ancak projemizde gereksiz olan ek sütunlara ihtiyaç duyulmamıştır.

- `clients.conf`: Localhost ve oluşturulan Docker ağ aralığı (172.16.0.0/12) yetkili NAS cihazı olarak tanımlanmış, ortak gizli anahtar (shared secret) ise `testing123` olarak belirlenmiştir.
- `sites-enabled/default`: Sistemin kalbini oluşturan bu konfigürasyonda; `authorize` bloğu REST modülünün `/authorize` ucuna giderek Auth-Type'ı API'den gelen değere (Rest) göre ayarlar. `authenticate` bloğunda REST modülü `/auth` ucuna giderek kimlik doğrulamayı tamamen API'ye devreder; API'den dönen HTTP durum koduna (200 OK veya 401 Reject) göre işlem kabul edilir. `accounting` bloğunda ise veritabanına doğrudan yazmak yerine yine REST modülü kullanılarak API entegrasyonu sağlanmıştır.

Docker Altyapısı

Tüm servisler tek bir `docker-compose.yml` dosyası üzerinde kurgulanmıştır. Sistemin dayanıklılığını artırmak için her servise özel sağlık kontrolleri (healthcheck) eklenmiştir: PostgreSQL için `pg_isready`, Redis için `redis-cli ping`, API için Python'ın `urllib` kütüphanesiyle `/health` uç noktası ve FreeRADIUS için yerleşik `radtest` komutu kullanılmıştır. Ayrıca `depends_on` ve `service_healthy` koşulları kullanılarak, veritabanı ve önbellek gibi alt servisler tam anlamıyla hazır olmadan API ve FreeRADIUS servislerinin başlatılması engellenmiştir.

Tanımlanan Docker volume (veri alanı) yapıları iki temel amaca hizmet etmektedir: `postgres_data` ve `redis_data` gibi isimlendirilmiş (named) volume'lar veritabanı ve önbellek verilerinin kalıcılığını sağlarken; FreeRADIUS konfigürasyonlarında kullanılan klasör bağlama (bind mount) yöntemi, ayarların konteyner dışında esnek bir şekilde yönetilmesine olanak tanımıştır.

4.Güvenlik Önlemleri ve Potansiyel Riskler

Sistem mimarisi tasarlanırken, siber güvenlik prensipleri (Savunma Derinliği - Defense in Depth) göz önünde bulundurulmuş ve her bir katmanda spesifik önlemler alınmıştır.

Alınan Güvenlik Önlemleri

- Kriptografik Şifreleme (Bcrypt): RADIUS sunucularında sıklıkla karşılaşılan en büyük zafiyet, şifrelerin veritabanında MD5 veya düz metin (cleartext) olarak tutulmasıdır. Bu projede, kimlik doğrulama yükü API katmanına çekilerek şifreler bcrypt algoritmasıyla güvence altına alınmıştır. Bcrypt'in barındırdığı `cost` (çalışma maliyeti) faktörü ve her şifreye özel üretilen rastgele salt değeri sayesinde, *brute-force* (kaba kuvvet) ve *rainbow table* saldırıları teknik olarak imkânsız hale getirilmiştir.

- Brute-Force ve DoS Koruması (Rate-Limiting): Kimlik doğrulama uç noktasına yapılabilecek saldırıları engellemek amacıyla Redis tabanlı bir hız sınırlandırma (rate-limit) mekanizması kurulmuştur. Aynı kullanıcı adı ile art arda 5 başarısız giriş denemesi yapıldığında, ilgili hesap 300 saniye (5 dakika) boyunca kilitlenerek yetkisiz erişim denemeleri ve kaynak tüketimi (DoS) engellenmiştir.
- SQL Injection Koruması: Veritabanı etkileşimlerinde asynccpg kütüphanesi tercih edilmiş ve tüm SQL sorguları dışarıdan alınan parametrelerle (\$1, \$2 yapısı) çalıştırılmıştır. Bu yaklaşım, dışarıdan gelebilecek zararlı SQL enjeksiyon (SQLi) saldırılarını tamamen önlemektedir.
- Ağ İzolasyonu (Network Segmentation): Sistemdeki bileşenler Docker üzerinde radius-net isimli kapalı bir ağda (bridge network) çalıştırılmıştır. Sadece FreeRADIUS'un 1812 (Auth) ve 1813 (Acct) portları dış dünyaya açılmış olup; PostgreSQL, Redis ve FastAPI servisleri dış ağ erişimine tamamen kapatılarak yanal hareket (lateral movement) riski minimize edilmiştir.

Potansiyel Riskler ve Geliştirme Önerileri

- PAP Protokolünün İletişim Zafiyeti: Projede kimlik doğrulama metodu olarak PAP kullanılmıştır. PAP, şifreleri ağ üzerinde zayıf bir şifrelemeyle (veya düz metin olarak) iletir. Gerçek bir senaryoda, istemci (supplicant) ile RADIUS sunucusu arasındaki bu trafiğin EAP-TTLS veya PEAP (802.1X) gibi güvenli tünellerle (TLS tabanlı) şifrelenmesi zorunludur.
- API İletişimi (Cleartext HTTP): FreeRADIUS ile FastAPI arasındaki haberleşme, izole Docker ağı içerisinde http://api:8000 üzerinden şifresiz yapılmaktadır. Dağıtık bir mimaride (API ve RADIUS'un farklı fiziksel sunucularda olduğu durumlarda), bu iletişimin HTTPS üzerinden sağlanması ve Servisler Arası Kimlik Doğrulama (mTLS) mekanizmasının kurulması gerekir.
- Statik Secret Key Kullanımı: NAS cihazı ile FreeRADIUS arasındaki iletişim testing123 gibi zayıf ve statik bir paylaşılan anahtar (shared secret) ile sağlanmaktadır. Üretim ortamında bu anahtarların kriptografik olarak güçlü (en az 32 karakter) seçilmesi ve HashiCorp Vault gibi güvenli bir "Secret Manager" üzerinden dinamik olarak yönetilmesi gerekmektedir.

5.Gerçek Dünya Uyarlamaları

Bu yapılan proje aşağıdaki gibi sektörlerde çeşitli amaçlarda kullanılabilir:

Örneğin sağlık sektöründe tıbbi cihazların izole edilmesi ve KVKK uyumluluğunun sağlanması , ISP (İnternet Servis Sağlayıcı) ortamlarında abone kullanımlarının takibi veya akıllı binalardaki IoT cihazlarının ağ düzeyinde sınırlandırılması gibi yüksek güvenlik gerektiren çeşitli sektörlerde doğrudan uygulanabilir niteliktedir.

Özellikle sistemin dinamik VLAN atama yeteneği, aşağıdaki iki temel senaryoda kritik bir rol oynamaktadır:

- Kurumsal Ağ Güvenliği ve Cihaz Segmentasyonu: Modern şirket ağlarında, şirket çalışanları ile dışarıdan gelen misafirlerin veya taşeronların aynı ağ kaynaklarına erişmesi büyük bir güvenlik zafiyetidir. Kurumsal ağ güvenliği kapsamında, çalışan ve misafir cihazlarının segmentasyonu bu sistem ile otomatik olarak sağlanabilir.
 - Uygulama: Ağa fiziksel (kablolu) veya kablosuz (802.1X Enterprise Wi-Fi) olarak bağlanan bir cihazın kullanıcısı, FastAPI motoru üzerinden doğrulanır. Eğer kullanıcı employee (çalışan) grubundaysa sistem Tunnel-Private-Group-Id: 20 özniteliğini dönerek kullanıcıyı iç sunuculara ve şirket intranetine erişimi olan VLAN 20 ağına dâhil eder. Eğer giriş yapan kişi guest (misafir) grubundaysa, cihaz izole edilmiş ve sadece internet erişimi olan VLAN 30 (Misafir Ağı) segmentine atanır. Böylece yetkisiz bir kişinin toplantı odasındaki bir ağ prizine bilgisayarını takarak şirket içi sunuculara ulaşması ağ düzeyinde (Layer 2) engellenmiş olur.
- Eğitim Kurumları ve Kampüs Ağ Yönetimi (Örn: eduroam yapısı): Üniversite kampüsleri; öğrenci, akademisyen ve idari personel gibi çok farklı profillerdeki binlerce kullanıcının aynı fiziksel ağı paylaştığı, yüksek trafikli ortamlardır. Bu ortamlarda öğrenci, akademisyen ve personel için farklı ağ politikaları uygulanması gerekmektedir.
 - Uygulama: Bu sistem kampüs ağına entegre edildiğinde, bir akademisyen ağına bağlandığında araştırma veritabanlarına ve idari sistemlere erişimi olan özel bir VLAN'a atanırken; bir öğrenci yalnızca genel internet ve öğrenci bilgi sistemine erişebildiği farklı bir VLAN'a yönlendirilebilir. Ayrıca, kampüs ağlarına dışarıdan yapılabilecek brute-force (deneme-yanılma) saldırılarına karşı, sistemimizde yer alan Redis tabanlı rate-limiting mekanizması devreye girerek kötü niyetli istekleri kaynağında keser. Buna ek olarak, Accounting (Hesap Yönetimi) altyapısı sayesinde hangi öğrencinin veya personelin hangi saatler arasında ağına bağlı kaldığı (Start/Stop paketleri) ve ne kadar veri aktardığı PostgreSQL üzerinde loglanır. Bu veriler hem 5651 sayılı yasa gereksinimleri (loglama) hem de adil kullanım politikalarının uygulanması için kullanılabilir.

6.Sonuç ve Öğrenilenler

Proje boyunca Docker, FreeRADIUS, FastAPI, PostgreSQL ve Redis teknolojileri entegre bir mikroservis mimarisi içerisinde kullanılmıştır. Geliştirme sürecindeki en zorlayıcı kısım FreeRADIUS konfigürasyonları olmuştur; protokol hata mesajlarının belirsizliği ve dokümantasyonun karmaşıklığı hata ayıklama (debug) sürecini uzatmıştır. Özellikle rlm_rest modülü ile FastAPI arasındaki JSON veri formatı uyumsuzluğunun çözülmesi ve authenticate bloğundaki dinamik Auth-Type yönetimi, RADIUS AAA mimarisinin çalışma mantığını kavramak adına projenin en değerli öğrenme noktaları olmuştur.

Mimari bir karar olarak şifre güvenliğinde bcrypt tercih edilmesi, kimlik doğrulama yükünün veritabanından alınarak tamamen FastAPI katmanına taşınmasını gerektirmiştir. Bu yaklaşım, FreeRADIUS'un yerleşik (native) olarak desteklemediği modern bir kriptografik algoritmayı kullanmayı mümkün kılmıştır. Alınan bu karar kod miktarını ve entegrasyon eforunu artırsa da, güncel siber güvenlik standartlarını yakalamak ve sistemin esnekliğini korumak adına atılmış son derece doğru bir mühendislik adımı olarak değerlendirilmektedir.

Tüm bunların yanı sıra AAA yapısı, RADIUS protokülü, FreeRADIUS, PostgreSQL ve Redis gibi yapılar da kişisel olarak bu projede daha derinine indiğim konular oldu. Özellikle RADIUS ve FreeRADIUS kavramları hakkında açıkçası pek bilgim olmadan başladığım bu ödevde hem öğrendim hem de öğrendiklerimi uygulamaya geçirdim. Açıkçası FreeRADIUS konfigürasyonu kısmı en çok uğraştıran kısım oldu. Sadece bu aşamda 13 tane başlıca hata not ettim. Sırasıyla permission hatası, clients.conf ve mods-enabled dosyalarında hatalar meydana geldi. Sonrasında FreeRADIUS tarafında birçok sorunla karşılaştım. Permission, sql, table, authorization, column pydantic access-request, auth-type gibi kısımlarda sıralı hatalar alsam da sonuç olarak bunları çözdüm ve hiç bilim olmadan başladığım bu alanda (FreeRADIUS ve RADIUS) birçok bilgi edindim. En son olarak da acoounting testinde radacct tablosuyla ilgili bir sorunla karşılaştım da bunu da sites-enabled/default içindeki düzenlemelerle çözdüm. Aldığım birçok not, ne yaptığımı anlamamda bana çok katkı sağladı. Kod yazımı kısımlarında yapay zeka araçlarını da kullandığımı belirtmekte bir sakınca görmüyorum. Neyin neden yapıldığı üzerine fikirlerim var ve not aldığım birçok şey de var. Özellikle net bir deadline olmaması kod kısmında ister istemez süreci hızlandıracak araçlara itiyor.

7.Kaynaklar ve Araçlar

1. FreeRADIUS Official Documentation & Wiki: RADIUS sunucusunun yapılandırılması, sanal sunucu (virtual server) mantığı ve özellikle rlm_rest ile rlm_sql modüllerinin entegrasyon detayları için temel referans olarak kullanılmıştır. (<https://wiki.freeradius.org/>)
2. IETF RFC 2865 & RFC 2866: Network Access Control mimarisinin temelini oluşturan RADIUS Kimlik Doğrulama (Authentication/Authorization) ve Hesap Yönetimi (Accounting) protokollerinin standartlarını ve paket yapılarını kavramak amacıyla incelenmiştir.

3. FastAPI & Pydantic V2 Documentation: Asenkron policy engine mimarisinin kurulması, RESTful uç noktaların (endpoints) oluşturulması ve FreeRADIUS'tan gelen karmaşık JSON veri tiplerinin model_validator kullanılarak ayrıştırılması süreçlerinde başvurulmuştur.
(<https://fastapi.tiangolo.com/>)
4. OWASP Password Storage Cheat Sheet: Kimlik doğrulama sürecinde şifrelerin MD5 yerine bcrypt ile güvence altına alınması ve güvenli kimlik doğrulama mimarisi kararlarının desteklenmesi için referans alınmıştır.
5. asyncpg Documentation: PostgreSQL veritabanı ile Python arasındaki asenkron iletişimi sağlamak, bağlantı havuzu (connection pool) oluşturmak ve parametrik ham SQL (raw SQL) sorguları yazmak için kullanılmıştır.
6. Redis Official Documentation: Başarısız giriş denemelerine karşı hız sınırlandırması (rate-limiting) mekanizmasının kurulması ve aktif oturumların in-memory (bellek içi) önbelleklenmesi işlemlerinde faydalanılmıştır.
7. NanoBananaPro: Görsel üretiminde kullanılmıştır.